01
WEDNESDAY
AUGUST

AUGUST 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 31 (213-152)

8 am
The Terminal :-
A text input and output environment.
9 am
It can be used :
- To install packages
- Git commands
10 am
- Programming related to terminal
- Speed / efficiency
11 am
- Access hidden items / files, internal softwares, OS
12 noon
• Command Line - Any interface that is used by entering textual commands (generally Windows Centric)
1 pm
• Terminal - This is a type of command line (generally Mac centric)
2 pm
• Console - A command-line interface used to work with the computer.
3 pm
• Shell - A program running on terminal
4 pm
• Bash - A popular shell on Mac OS / Linux
• Z - Shell - Another shell (default)
5 pm
↳ All these terms are interchangeably used and its normal.
6 pm
⇒ Install Git - Bash
Basic Commands :-
• ls → list files
(shows all folder / files in given / default directory)
[~ → Home Directory] [• / → root Directory]
• pwd → print working directory
(where am I?)
• clear → clears screen
[upward and downward keys for older code / commands]
2018
[tab to auto complete]
• git • --version → shows current version git using.
• git → shows all git command

2018 SEPTEMBER
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30

WK 31 (214-151)

02
THURSDAY
AUGUST

Navigation :-
8 am
Inside and Outside Directories
• cd → change directory
e.g. cd Desktop (Single Jump)
cd Desktop / Delt / CSS (Multiple Jump)
10 am
• cd .. → back button
e.g. cd .. (Single Back)
11 am
cd ../.. (Multiple Back)
12 noon
Paths :-
Absolute & Relative
1 pm
Relative → Depends on current folder / directory
e.g. cd Desktop / Delt (Relative Path)
2 pm
Absolute → Do not depend on anything, always same
e.g. cd / Users / ritank / Desktop / Delta (starts with /)
3 pm
cd / → root directory
4 pm
cd ~ → home directory
5 pm
Making Directories :-
• mkdir → makes directory / folder
6 pm
e.g. mkdir NewF (The directory should not have same folder)
(Relative and absolute paths can be used)

Flags :-
Flags are characters that we pass with commands to modify their behaviour.
• man → Manual Command
Shows manual for given command which also consists of all the avilable flags which can be used with that command. Press q to quit.
2018
e.g. man ls (gives info about ls command)

03
FRIDAY
AUGUST

AUGUST 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 31 (215-150)

8 am commands with flags eg.:
ls -l
9 am ls -a
ls -la
10 am

Touch command :-
11 am • touch → used to create files / used to change file ~~of~~ access and modifications times but if file
12 noon does not exist, it may created with default permissions.
1 pm e.g. touch file
touch index.html
2 pm touch abc.txt
3 pm

Deleting Files & Folders :-
4 pm This is a sensitive command, must be used carefully because if any file / folder gets deleted through this
5 pm it will be gone forever, and will not even be found in recycle bin and cannot be retrived.
6 pm • rm → remove files e.g. rm index.html style.css
• rmdir → removes strictly empty folders only
• rm -rf → removes any folders
(-rf → recursive force → removes with force)

2018

2018 SEPTEMBER
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30

WK 31 (216-149)

04
SATURDAY
AUGUST

Git & Github :-
8 am Git - Free & Open Source Version Control System which is a tool that helps to track changes in code.
9 am It tracks history and help to collaborate.
It is very fast and scalable.
10 am Github - It is a website where we host repositories online. It is based on git only.
11 am

README.md → (markdown) About / Overview of the project
12 noon commit → change (two step process) (tracks all changes) kind of
add
1 pm ↓
commit
2 pm

Using Git :-
3 pm • command line (most popular) (No limitations)
• IDE / Code Editors (like VS Code)
4 pm • Graphical User Interface (like Git Kraken)

5 pm Configuring Git :-
git config --global user.name "Ritank Jaikar"
6 pm git config --global user.email "ritank.jaikar@gmail.com"
(other options → local, system)
check → git config --list

SUNDAY 05

Clone command :- cloning a repository on our local machine / VS Code.
git clone <link>
→ github repo link

2018

06
MONDAY
AUGUST

AUGUST 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 26
27 28 29 30 31

WK 32 (218-147)

8 am Status Command :- displays the state of code.
git status

9 am File Status Life Cycle

10 am untracked unmodified modified staged

11 am edit the file → stage the file →
add the file →
← remove the file
commit ←

12 noon

1 pm

2 pm

VS code symbol :-

(V) 3 pm Untracked → Any new file is untracked file if not added. So any modification will not be tracked.

( ) 4 pm Unmodified → No modification done, ~~file is~~ but file is added and trackable. Does not show on status. (Also committed)

(M) 5 pm Modified → If file is changed ~~but not a~~, also previously added, but not added again after modification.

(A) 6 pm Staged → If file is both added ~~and added~~ previously or added after modification, but not committed. (after ~~modification or when newly added~~)

Usually git status → shows untracked, modified & staged files. Other leftout files are unmodified/committed ~~(untracked)~~.

• add → adds new or changed files in your working directory to the Git staging area.
untracked/modified files → staged files
git add <file name>
git add . → adds all files) (All files will be staged)

2018

2018 SEPTEMBER
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30

WK 32 (219-146)

07
TUESDAY
AUGUST

8 am • commit → it is the record of change. Only staged files can be committed.
git commit -m "some message" → this message is mendatory & must be meaningful)

9 am staged files → commited/~~unstaged~~

10 am • push → upload repo content to remote repo. (Github)
git push origin main → (github account must be same logged in)

11 am (origin → location of github repo, main → branch, both can be ignored after first push)

12 noon add → commit → push → Project git workflow)

1 pm

2 pm

3 pm • init - used to create a new git repo. (in local)
• git init

4 pm • git remote add origin <link> → to add link local git repo to github using link)

5 pm • git remote -v → To verify remote)
• git branch → To check branch)

6 pm • git branch -M main → To rename branch) (from master to main)
• git push origin main → To finally push/upload)

• git push -u origin main → to use origin main only once that is first ~~time~~ time)
↳ (upstream)
From second time ~~onwards~~ onwards →
git push → only this will work)

• git ~~commit~~ commit -am "some message"
used to add and commit together. (do not add untracked files)

2018

08 WEDNESDAY AUGUST

AUGUST 2018

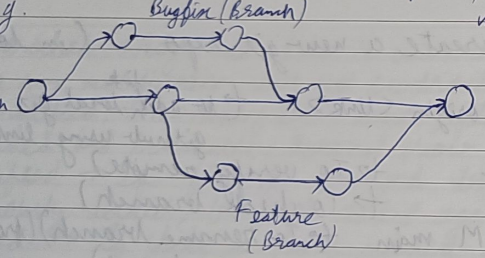
WK 32 (220-145)

8 am Workflow :- (till now)

| 9 am | Github | Local Git |
|---|---|---|
| | ⇓ | ⇓ |
| 10 am | code changes | code changes |
| | ↓ | ↓ |
| 11 am | commit | add |
| | | ↓ |
| 12 noon | | commit |
| | | ↓ |
| 1 pm | | push |

2 pm Git Branches :- It is a new/seperate/clone version of the main repositary. It use can use to add new feature, fix bugs, etc. seperately.

e.g.




- git branch → to check all branches
- git branch -M <new name> → to rename branch
- git checkout -b <new branch name> → to create new branch
- git checkout <branch name> → to navigate
- git branch -d <branch name> → to delete branch
- git push --set-upstream origin <branch name> → to push the branch

2018 → These branches create tree structure.

2018 SEPTEMBER

WK 32 (221-144)

09 THURSDAY AUGUST

Merging Code :-

8 am
- git dif <branch name> → to compare commits, branches, files & more
9 am
- git merge <branch name> → to merge 2 branches

OR

10 am Create a PR (Pull Request on Github)

11 am Pull Request :-

It lets you tell others about changes you've pushed to a branch in a repositary on Git Hub. Then we / manager / head developer can see the request, and then merge pull request. This is commit itself. And always remember to add meaningful comments.

1 pm
2 pm
- git pull origin main (→ on local)

used to fetch and download content from remote repo and immediately update the local repo to match the content.

3 pm
4 pm

5 pm Merge Conflicts :-

An event that takes place when Git is unable to automatically resolve difference in code between two commits. (when pull request to merge two branches)

conflict

6 pm We can manually handle this, either of both on local (VS Code) or GitHub,

Fixing Mistakes :-

Case 1: staged changes (Only add)
- git reset <file name> → for single file
- git reset → for all file, reset to the staged changes

Now only modified left.

2018

q → to quit

10 FRIDAY AUGUST

WK 32 (222-143)

AUGUST 2018

| M | T | W | T | F | S | S | M | T | W | T | F | S | S |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 |
| 27 | 28 | 29 | 30 | 31 | | | | | | | | | |

2018 SEPTEMBER

| M | T | W | T | F | S | S | M | T | W | T | F |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | 1 | 2 | 3 | 4 | 5 | 6 | |
| 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 | | | | | |

8 am

Case 2: Commited changes (for ~~many~~ one commits)

- git reset HEAD~1 → Resets to previous commit, now only modified left.

Head pointer ~~(for~~ points to commits

9 am

10 am

11 am

Case 3: Committed changes (for many commits)

- git log → shows all comits hash with comments in reverse order (new to old)
- git reset <comit hash> → ~~resets~~ resets to selected commit, ~~no~~ modified left.
- git reset --hard <commit hash> → resets and all the changes/gone modified

12 noon

1 pm

2 pm

3 pm

Forking :-

4 pm

A fork is a new repository that shares and visibility settings with the original "upstream" repository.

5 pm

Fork is a rough copy of other's repository.

6 pm

OOPS Conce

OOP is prog objects, w of fields of objects access and

Topics :-

- prototype
- Keywords

Object Pr Prototy objects It is t objects having Every which i an obj makin ends its

e.g. Th ha